SIMATS ENGINEERING
ASSESSMENT TOOL 3: Mock Interview

COURSE NAME:	Cloud Computing and Big Data Analytics	**COURSE CODE:**	CSA15

COURSE OUTCOME COVERED
CO5: *Design big data processing systems using the Hadoop ecosystem including HDFS, MapReduce, and YARN.* (BL5)

Dave's Taxonomy: Articulation (Level 4) **Total Marks: 25**
Question Count: 10

Code of Conduct
I ____________________ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action. Signature of the student: ____________________

Assessment Tasks Mock Interview
Answer the following interview-oriented questions clearly and concisely. Support your responses with architecturelevel reasoning wherever appropriate.

1. Explain the difference between HDFS and traditional file systems.
2. Why is replication important in distributed storage systems?
3. How does MapReduce divide work between map and reduce phases?
4. What role does YARN play in large-scale Hadoop processing?
5. Differentiate NAS and SAN in an enterprise data environment.
6. What is the purpose of ETL in a big data pipeline?
7. How does Apache NiFi help in data integration workflows?
8. What are the key failure points in a Hadoop cluster and how would you handle them?
9. How would you design a secure data ingestion pipeline for a large organization?
10. How would you explain a scalable Hadoop-based processing system to a non-technical manager?

Mock Interview Rubric

Communication	20%	Answers are confident, precise, and professional	Mostly clear communication	Moderate clarity	Weak communication

Practical Awareness	20%	Connects concepts to real-world implementation well	Some practical relevance	Limited practical relevance	No practical linkage
Confidence and Professionalism	15%	Highly confident and professional	Generally professional	Moderately professional	Low confidence and professionalism

1. Explain the difference between HDFS and traditional file systems.

HDFS (Hadoop Distributed File System) is designed to store very large amounts of data across multiple computers, while traditional file systems usually store data on a single computer or storage server.

Key differences:

1. **Storage:** HDFS distributes files across many machines, whereas traditional file systems generally use local storage.
2. **Data size:** HDFS is suitable for **TBs or PBs of data** and large files.
3. **Fault tolerance:** HDFS uses **replication** to protect data from hardware failures.
4. **Scalability:** HDFS can be expanded by adding more machines to the cluster.
5. **Access:** HDFS is optimized for **high-throughput, sequential access** rather than frequent small-file updates.

Example: If a 1 GB file is stored in HDFS, it can be divided into blocks and distributed across several machines.

2. Why is replication important in distributed storage systems?

Replication means maintaining multiple copies of the same data on different machines.
It is important because:

1. **Fault tolerance:** If one server fails, another copy of the data is available.
2. **Data availability:** Users can continue accessing data even when some machines are unavailable.
3. **Reliability:** It protects data against hardware and disk failures.
4. **Performance:** Multiple copies can allow data to be read from a nearby or less busy machine.
5. **Recovery:** Failed or lost copies can be recreated from healthy replicas.

Example: HDFS commonly uses a replication factor of **3**, meaning three copies of a data block can be stored on different nodes.
Thus, replication improves **reliability, availability, and fault tolerance**.

3. How does MapReduce divide work between map and reduce phases?

MapReduce is a distributed programming model used to process large datasets.
It divides the work into two major phases:

1. **Input:** Large data is divided into smaller blocks and distributed among machines.
2. **Map phase:** Each mapper processes its input and produces **key-value pairs**.
3. **Shuffle and Sort:** The intermediate key-value pairs are grouped according to their keys.
4. **Reduce phase:** Reducers process each group and combine or summarize the values.
5. **Output:** The final results are stored in the distributed file system.

Example: For counting words:

- **Map:** (hello, 1), (world, 1), (hello, 1)
- **Reduce:** (hello, 2), (world, 1)

Therefore, MapReduce enables large datasets to be processed **in parallel across many machines**.

4. What role does YARN play in large-scale Hadoop processing?

YARN (Yet Another Resource Negotiator) is the resource management and job scheduling component of Hadoop.

Its main roles are:

1. **Resource management:** Allocates CPU, memory, and other resources to applications.
2. **Job scheduling:** Decides when and where jobs should run.
3. **Cluster utilization:** Helps use cluster resources efficiently.
4. **Application management:** Manages the execution of different applications.
5. **Scalability:** Allows multiple processing frameworks to run on the same Hadoop cluster.

YARN mainly consists of the **ResourceManager**, **NodeManagers**, and an **ApplicationMaster** for each application.

Thus, YARN acts like a **traffic controller**, managing different jobs and resources in a large Hadoop cluster.

5. Differentiate NAS and SAN in an enterprise data environment.

NAS (Network Attached Storage) and **SAN (Storage Area Network)** are both used for centralized enterprise storage, but they work differently.

NAS	SAN
Provides **file-level storage**	Provides **block-level storage**
Accessed using network file protocols	Accessed using storage protocols
Usually easier to configure	More complex to configure
Suitable for file sharing and backups	Suitable for databases and high-performance applications
Generally lower cost	Generally more expensive

Example:

NAS can be used by employees to share documents, while SAN can be used by a company's database servers.

Therefore, **NAS is mainly file-oriented**, whereas **SAN provides high-performance block storage**.

6. What is the purpose of ETL in a big data pipeline?

ETL stands for **Extract, Transform, Load**. It is used to prepare data before storing or analyzing it.

1. **Extract:** Collects data from different sources such as databases, applications, files, and APIs.
2. **Transform:** Cleans and changes the data into a suitable format.
3. **Load:** Stores the processed data in a data warehouse, data lake, or other storage system.
4. **Data quality:** Removes duplicate, incorrect, or incomplete data.
5. **Analysis:** Makes data ready for reporting, analytics, and machine learning.

Example: Customer data collected from different branches can be extracted, cleaned, converted into a common format, and loaded into a central data warehouse.

Thus, ETL ensures that data is **clean, consistent, and ready for analysis**.

7. How does Apache NiFi help in data integration workflows?

Apache NiFi is a data integration and data-flow automation tool. It helps move and process data between different systems.

Its important features are:

1. **Data movement:** Transfers data between databases, files, APIs, and other systems.
2. **Flow management:** Allows users to visually design data flows.
3. **Data transformation:** Can modify and convert data during processing.
4. **Real-time processing:** Supports continuous data movement.
5. **Monitoring:** Provides tracking and monitoring of data flow.

NiFi also provides **data provenance**, which helps track where data came from and where it has gone.

Therefore, NiFi makes large-scale data integration **easier, visual, automated, and manageable**.

8. What are the key failure points in a Hadoop cluster and how would you handle them?

A Hadoop cluster contains many machines, so failures can occur at different levels.

Key failure points and solutions:

1. **DataNode failure:** HDFS replication provides another copy of the data.
2. **NameNode failure:** Use **NameNode High Availability** with multiple NameNodes.
3. **Disk failure:** Replace failed disks and allow data blocks to be re-replicated.
4. **Network failure:** Use redundant network connections and switches.
5. **ResourceManager failure:** Configure YARN High Availability.

Additional measures include:

- Monitoring cluster health continuously.
- Maintaining proper replication levels.
- Taking regular backups.
- Using automatic recovery mechanisms.

Thus, **replication, high availability, monitoring, and automatic recovery** help keep the Hadoop cluster running even when failures occur.

9. How would you design a secure data ingestion pipeline for a large organization?

A secure data ingestion pipeline should protect data from the point where it enters the system until it is stored and processed.

A suitable design is:

Data Sources → Secure Ingestion → Validation → Encryption → Storage → Processing Important security measures include:

1. **Authentication:** Verify users and systems sending data.
2. **Encryption:** Use encryption for data both **in transit and at rest**.
3. **Data validation:** Check incoming data for invalid or malicious content.
4. **Access control:** Give users only the permissions they require.
5. **Monitoring and auditing:** Record access and data movement activities.

Tools such as secure APIs, certificates, access-control mechanisms, and audit logs can be used.

Thus, a secure pipeline provides **confidentiality, integrity, authentication, and controlled access** to organizational data.

10. How would you explain a scalable Hadoop-based processing system to a nontechnical manager?

A Hadoop-based system can be explained using a simple example.

Imagine a company receives **millions of customer transactions every day**.

1. Hadoop stores the large amount of data by **dividing it across many computers**.
2. Multiple copies of data are maintained so that data is not lost if a computer fails.
3. Processing tasks are divided into smaller tasks and performed **simultaneously on many computers**.
4. YARN manages the available computer resources and distributes work efficiently.
5. When the company gets more data, **new computers can be added** to increase storage and processing capacity.

Simple analogy: Hadoop is like having a large team of workers. Instead of one person doing a huge job, the work is divided among many workers, allowing the company to process more data quickly.